

Performance Analysis of CUDA-Based Parallelization Strategies for a Shallow Neural Network

Bouameur Besmala Aicha

Djafour Touria

Hammou Manel

Kellou Lylia Ines

Messaoud Amal

Abstract—Graphics Processing Units are widely used to accelerate machine learning workloads due to their parallel computing capabilities. This paper analyzes CUDA-based parallelization strategies for training a shallow neural network. A reference GPU implementation is first examined and used as a performance baseline for the forward and backward propagation phases. An alternative strategy is then evaluated to improve computational efficiency. Experimental results compare execution time and scalability, highlighting the impact of parallelization choices and memory usage on GPU performance.

Index Terms—CUDA, GPU computing, shallow neural network, parallelization, performance analysis

I. INTRODUCTION

Matrix multiplication is the dominant operation in both the forward and backward propagation of shallow neural networks. Consequently, training performance is largely determined by how efficiently these linear algebra kernels are executed. GPUs provide a highly parallel architecture for such workloads, but performance is strongly influenced by kernel design choices, including thread mapping and memory access patterns.

In this work, we use a shallow neural network as a case study to examine how a reference CUDA implementation maps matrix computations to the GPU and where its performance limitations arise. We then evaluate an alternative implementation designed to improve efficiency by reducing memory overhead and increasing data reuse. The study reports timing results across multiple dataset sizes to characterize scalability trends.

II. SHALLOW NEURAL NETWORK MODEL

A. Architecture

We consider a shallow neural network composed of a single hidden layer. The input dimension is 32 features, followed by a hidden layer of 256 neurons with ReLU activation, and an output layer producing a single scalar value for a regression task. This simple architecture was chosen to facilitate the analysis of different GPU parallelization strategies for both forward and backward propagation. The network is trained using mini-batch stochastic gradient descent with a batch size of 256, a learning rate of 0.002, and 100 epochs, using mean squared error (MSE) as the loss function.

B. Key Characteristics and Assumptions

The proposed network is intentionally lightweight in order to isolate the impact of parallelization strategies from architectural complexity. Its main computational workload arises from matrix multiplications, which represent the primary performance bottleneck and the main target for GPU acceleration.

All experiments rely on synthetically generated data, with input values uniformly sampled from the range $[-1, 1]$ and additive Gaussian noise applied to the target output. This choice enables controlled experimentation and ensures reproducibility. The output layer contains a single neuron, assuming a univariate regression setting.

Hyperparameters such as the number of hidden neurons, learning rate, batch size, and number

of epochs are fixed throughout the experiments, without hyperparameter tuning. This design choice allows for a fair and consistent comparison between different parallelization strategies. Finally, the use of ReLU activation is assumed to be sufficient for modeling the non-linear relationships in the synthetic dataset, while maintaining computational efficiency.

III. ANALYSIS OF THE REFERENCE PARALLELIZATION STRATEGY

This section analyzes the reference parallelization strategies used for accelerating the training of a shallow neural network: row-wise multi-threading using pthreads, and fine-grained parallelization using CUDA. We evaluate computational costs, architectural considerations, and limitations of each approach.

A. Pthreads Parallelization

The reference strategy initially uses pthreads to parallelize matrix multiplication, which is the main computational bottleneck in both forward and backward propagation. The algorithm can be summarized as:

- 1) Divide the rows of the output matrix evenly among T threads.
- 2) Each thread computes all columns for its assigned rows:

$$C[i, j] = \sum_k A[i, k] \cdot B[k, j]$$

- 3) Synchronize threads and assemble the final output matrix.

This strategy accelerates computation, particularly for larger matrices. However, thread management overhead increases with the number of threads, leading to **diminishing returns** beyond a certain threshold. Performance is also influenced by memory access patterns and synchronization. Empirically, for small matrices, additional threads may provide minimal or even negative speedup due to overhead, whereas larger matrices benefit more substantially.

B. CUDA Parallelization

The GPU reference strategy accelerates training using CUDA, mapping each element of the output matrix to a single thread. Threads are organized in two-dimensional blocks and grids matching the output matrix dimensions. The computation follows the standard forward and backward propagation:

- **Forward pass:** Hidden pre-activations $Z_1 = X \cdot W_1$ are computed, followed by element-wise ReLU activation $H = \text{ReLU}(Z_1)$. Output predictions are obtained via $Y_{\text{pred}} = H \cdot W_2$.
- **Backward pass:** Gradients are computed for the output and hidden layers using matrix multiplications, applying the ReLU derivative on the hidden layer. Weight updates are performed using stochastic gradient descent.

Each thread executes the dot product for a single output element:

```
int row = blockIdx.y * blockDim.y + threadIdx.y;
int col = blockIdx.x * blockDim.x + threadIdx.x;

if (row < A_rows && col < B_cols) {
    float value = 0.0f;
    for (int k = 0; k < A_cols; k++)
        value += A[row * A_cols + k] * B[k * B_cols + col];
    C[row * B_cols + col] = value;
}
```

Input and output matrices are allocated in device memory, with explicit host-device transfers. Each thread repeatedly loads elements from global memory to compute the dot product, leading to limited data reuse and making the strategy memory-bound for certain matrix sizes. No tiling or shared-memory optimizations are applied in this baseline.

C. Computational Costs and Limitations

- **Pthreads:** Performance improves as threads are added, but thread management overhead can offset gains when the number of threads becomes large. Diminishing returns are observed beyond an optimal thread count, and memory access patterns and synchronization overhead affect scalability.
- **CUDA:** GPU execution achieves significant speedups due to massive parallelism. However, the fine-grained mapping and repeated

global memory accesses limit the computation-to-memory ratio. Kernel launch overheads and host-device memory transfers also add costs. The lack of memory-reuse strategies, such as tiling, reduces efficiency for intermediate-sized matrices.

D. Summary and Baseline Performance

Both strategies outperform sequential execution. Pthreads provides moderate speedup, particularly for larger matrices, whereas CUDA demonstrates superior scalability and performance due to massive parallelism. These baseline implementations provide a reference for evaluating alternative strategies that aim to optimize memory reuse and improve computation-to-memory ratio. The table below summarizes the baseline execution times for the reference parallelization strategies across datasets of different sizes. It highlights the performance improvements achieved with multi-threading using pthreads and GPU acceleration using CUDA, providing a clear comparison of scalability and speedup for small, medium, and large workloads.

TABLE I
BASELINE EXECUTION TIMES (S) OF THE REFERENCE
PARALLELIZATION STRATEGIES FOR DIFFERENT DATASET SIZES.

Dataset Size	Batches	Pthreads (s)	CUDA (s)
Small	1	0.83	0.45
Medium	10	7.44	2.59
Large	100	74.00	22.22

E. Profiling of the Reference CUDA Implementation

To better understand the performance limitations of the reference parallelization strategy, GPU profiling was conducted using an NVIDIA Tesla T4 GPU (40 SMs, 1.59 GHz, 15.64 GB memory) using NVIDIA Nsight Systems. The objective of this analysis is to identify execution bottlenecks, evaluate kernel efficiency, and analyze memory transfer behavior during the training process.

1) *Kernel Execution Analysis:* Profiling results show that the reference implementation relies on a single CUDA kernel, `mat_mult_kernel`, which is invoked repeatedly during training. For the large dataset configuration, the kernel is launched 50,000

times, with an average execution time of approximately 6.9 μ s per invocation. Despite the significant increase in the number of kernel launches, the average execution time remains nearly constant across medium and large datasets, indicating good scalability and efficient computation at the kernel level.

As illustrated in Fig. 1, the kernel execution achieves 100% occupancy on the hardware, yet it occupies a relatively small portion of the overall timeline compared to the surrounding API activity.

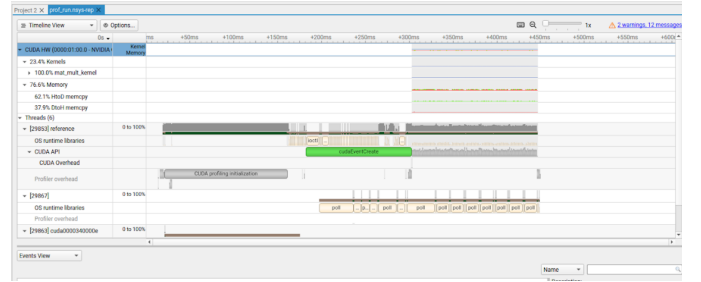


Fig. 1. Timeline detail of `mat_mult_kernel` showing 100% GPU utilization.

The total GPU kernel execution time represents only a small fraction of the overall training time, demonstrating that the kernel itself is not a performance bottleneck.

2) *CUDA API Overhead:* In contrast, profiling of CUDA API calls reveals a substantial overhead associated with memory management and data transfers. For the large dataset, `cudaMemcpy`, `cudaMalloc`, and `cudaFree` account for more than 85% of the total CUDA API execution time. These operations are invoked at every training iteration, resulting in frequent memory allocations, deallocations, and host-device data transfers.

Furthermore, synchronization calls such as `cudaDeviceSynchronize` contribute additional overhead, causing the CPU to block while waiting for GPU operations to complete. This behavior is captured in Fig. 2, where the macro-scale view reveals a pattern of CPU threads entering wait states.

3) *Memory Transfer Behavior:* Memory transfer analysis indicates a large number of small and synchronous data transfers between the host and the device. For the large dataset, more than 8.8 GB of data are transferred from host to device and

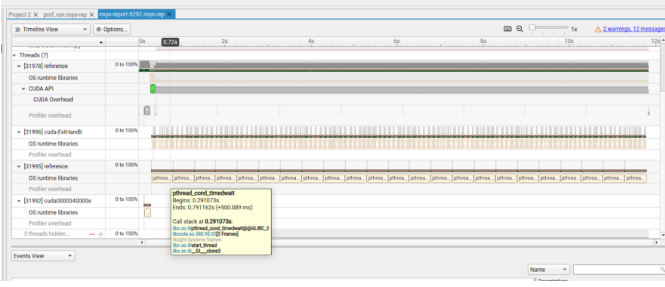


Fig. 2. Macro-view profile illustrating CPU blocking and synchronization wait states.

5.6 GB from device to host over the course of training. Although each transfer is relatively small, their high frequency significantly degrades overall performance.

This transfer pattern leads to poor overlap between computation and communication, effectively limiting the benefits of GPU acceleration.

To better quantify the observations obtained from Nsight Systems and Nsight Compute, Table II summarizes the execution time distribution of the reference CUDA implementation for a representative medium-sized input.

TABLE II
EXECUTION TIME BREAKDOWN OF THE REFERENCE CUDA IMPLEMENTATION (LARGE INPUT SIZE)

Component	Measured Time (s)	Observation
CUDA kernel execution (mat_mult_kernel)	0.343	Kernel execution is negligible compared to total training time
CUDA memory transfers (H2D + D2H)	1.338	Frequent small host-device and device-host transfers degrade performance
CUDA API overhead (malloc/free/sync, etc.)	8.654	Dominated by repeated cudaMalloc, cudaFree, cudaMemcpy, and synchronization calls
Total training time	11.438	Includes all CPU-GPU interactions and kernel execution

4) *Discussion*: Overall, profiling results demonstrate that the reference CUDA implementation is not compute-bound but rather dominated by memory management and data transfer overheads. While

the GPU kernel exhibits good performance and scalability, the fine-grained offloading strategy and repeated memory operations severely limit efficiency. These observations motivate the proposed parallelization strategy, which aims to reduce CUDA API overhead, minimize host-device data transfers, and improve overall GPU utilization.

IV. PROPOSED PARALLELIZATION STRATEGY

To address the limitations identified in the reference implementation, we propose a comprehensive optimization strategy consisting of three complementary techniques: tiled matrix multiplication, full GPU offloading, and kernel fusion. Each technique targets specific performance bottlenecks while maintaining numerical correctness.

A. Tiled Matrix Multiplication Optimization

To improve the performance of matrix multiplication on the GPU, a tiling-based optimization strategy was adopted. Tiling aims to reduce global memory access latency by exploiting fast on-chip shared memory and improving data reuse across threads within the same block.

In the naive CUDA implementation, each thread computes one element of the output matrix by repeatedly accessing global memory to load elements from the input matrices. Although this approach exposes massive parallelism, it suffers from redundant memory accesses, which significantly limits performance for large matrices.

The tiled approach partitions the input matrices into square submatrices (tiles) of size $T \times T$. Each thread block is responsible for computing one tile of the output matrix. During execution, tiles of the input matrices are cooperatively loaded from global memory into shared memory. Once loaded, all threads within the block reuse the shared data to compute partial dot products, thereby reducing the number of global memory transactions.

Let $A \in \mathbb{R}^{m \times n}$, $B \in \mathbb{R}^{n \times p}$, and $C \in \mathbb{R}^{m \times p}$. The output element C_{ij} is computed as:

$$C_{ij} = \sum_{k=0}^{n-1} A_{ik} \cdot B_{kj}$$

Using tiling, this summation is decomposed into smaller partial sums over tiles:

$$C_{ij} = \sum_{t=0}^{\lceil n/T \rceil - 1} \sum_{k=0}^{T-1} A_{i, tT+k} \cdot B_{tT+k, j}$$

Each iteration loads one tile from matrix A and one tile from matrix B into shared memory. A synchronization barrier (`__syncthreads()`) ensures that all threads have completed the loading phase before computation begins. After computing the partial result for the current tile, threads synchronize again before proceeding to the next tile.

1) *Memory Access Optimization:* The primary bottleneck addressed by tiling is the high latency and low reuse of global memory accesses. In the naive implementation, each element of matrices A and B is repeatedly fetched from global memory by multiple threads, leading to memory bandwidth saturation and underutilization of the GPU's arithmetic units.

The key idea of tiling is to restructure the computation so that data fetched from global memory is reused as much as possible before being evicted. By loading contiguous blocks of data into shared memory, memory accesses become coalesced and redundant loads are eliminated.

This approach increases arithmetic intensity by allowing each loaded value to participate in multiple floating-point operations. As a result, the computation shifts from being memory-bound toward being compute-bound, which is better aligned with GPU architecture.

2) Implementation Details:

- **Grid and Block Dimensions:** Each thread block is configured as a two-dimensional block of size `TILE_SIZE × TILE_SIZE` using `dim3 dimBlock(TILE_SIZE, TILE_SIZE)` although limited at a maximum of 1024 threads per block according to the hardware limitations. The grid dimensions are computed as

$$\text{dimGrid} = \left(\left\lceil \frac{B.\text{cols}}{TILE_SIZE} \right\rceil, \left\lceil \frac{A.\text{rows}}{TILE_SIZE} \right\rceil \right)$$

- **Memory Layout Changes:** Input matrices are accessed in row-major order to ensure coalesced global memory reads during tile loading.

- **Shared Memory Usage:** Two shared memory buffers are allocated per block to store tiles of matrices A and B .
- **Registers:** Partial sums are accumulated in registers to minimize memory traffic.
- **Synchronization Points:** Two `__syncthreads()` calls are used per tile iteration—one after loading tiles and one after computing partial results.

3) *Expected Impact and Trade-offs:* This optimization significantly reduces global memory traffic and improves cache efficiency. By increasing data reuse and ensuring coalesced accesses, execution time is reduced, especially for large matrices where memory bandwidth is the dominant constraint.

Larger tile sizes increase shared memory usage per block, which can reduce occupancy. Excessive register usage may also limit the number of concurrently active warps. Therefore, tile size must be carefully tuned to balance reuse and parallelism.

4) *Performance Analysis:* The tiling optimization was implemented in the CUDA kernel `_Kernel_mat_mult_tiled` to address the memory bottleneck. The kernel utilizes `__shared__` memory as a high-speed L1 cache. The process follows a collaborative loading pattern where all threads within a block cooperate to load a portion of data into shared memory tiles.

The impact of tile size was tested across three dataset scales: Small, Medium, and Large. The results are summarized in Table III.

TABLE III
GPU EXECUTION TIME (MS) FOR DIFFERENT TILE SIZES

Input Size	Tile 8	Tile 16	Tile 32	Tile 64
Small	2321	2323	2330	-
Medium	-	23068	23514	22722
Large	-	230885	232759	229877

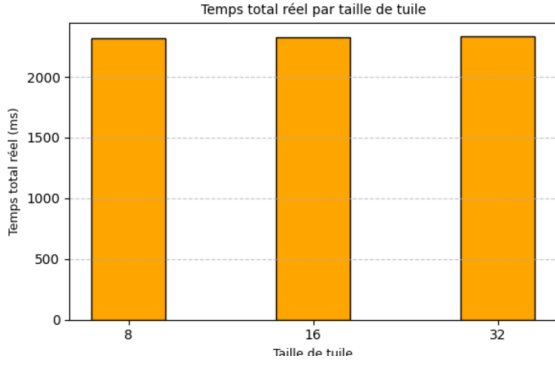


Fig. 3. Total execution time for Small dataset ($T = 8, 16, 32$)

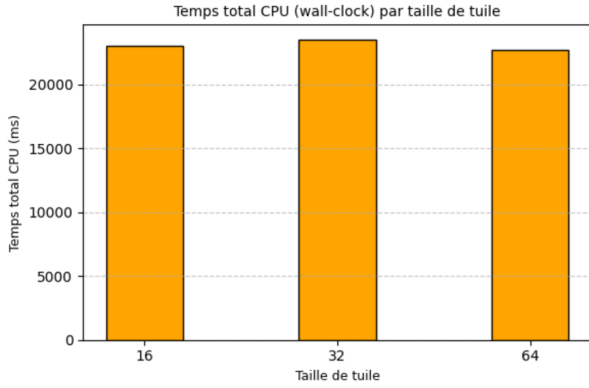


Fig. 4. Total CPU/Wall-clock time for Medium dataset ($T = 16, 32, 64$)

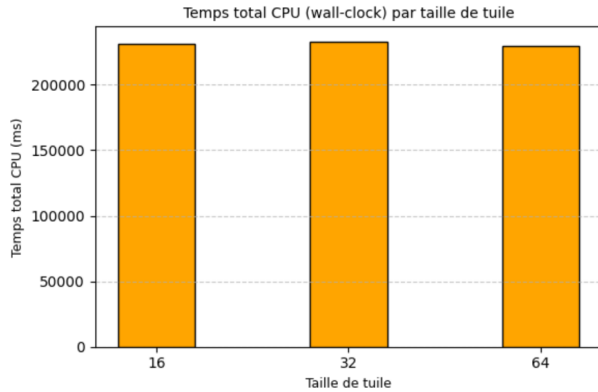


Fig. 5. Total CPU/Wall-clock time for Large dataset ($T = 16, 32, 64$)

5) *Convergence and Precision:* Regardless of the tile size used, the model's convergence remains stable. Figure 6 illustrates that the Mean Squared Error (MSE) drops sharply within the first few epochs and reaches a consistent plateau for all tested tile configurations, confirming that tiling optimizes speed without sacrificing numerical accuracy.

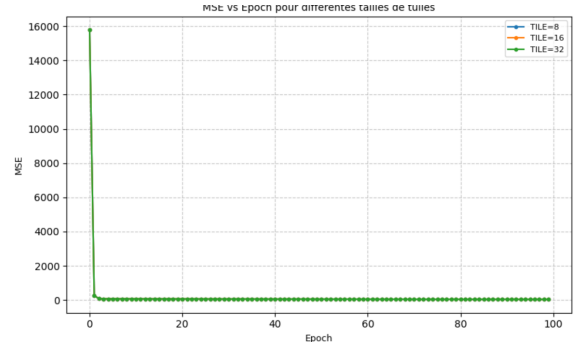


Fig. 6. MSE vs Epoch for different Tile Sizes

6) *Interpretation and Conclusion:* The experimental data reveals a nuanced relationship between tile size and performance:

- **Small Dataset:** The lowest execution time was achieved with $T = 8$ (2321.14 ms). At this scale, the overhead of managing larger shared memory tiles and the synchronization of more threads ($T = 32$) slightly outweighs the benefits of data reuse.
- **Medium and Large Datasets:** For larger scales, $T = 64$ emerged as the most efficient configuration (22721.70 ms for Medium and 22987.84 ms for Large). This suggests that as the data volume grows, maximizing the reuse of data within shared memory becomes the critical factor in overcoming the global memory bandwidth bottleneck.
- **Consistency:** The results for $T = 32$ were consistently slightly slower than $T = 16$ or $T = 64$ across the larger sets, indicating a potential "sweet spot" in occupancy or warp scheduling that favors $T = 64$ on this specific hardware.

In conclusion, tiling effectively hides memory latency. While smaller tiles are sufficient for small matrices, scaling the tile size (up to hardware limits) is essential for maintaining high throughput as dataset complexity increases.

B. Full GPU Offloading Optimization (GPU-Native Execution)

An important architectural optimization was applied by moving the entire training workflow to the GPU. In the initial hybrid implementation, parts of the neural network pipeline—such as intermediate

matrix operations, activation functions, and gradient computations—were still executed on the CPU. This resulted in frequent host-device memory transfers and synchronization overhead, which significantly limited performance.

The GPU-native optimization consists of offloading all computational stages of the forward and backward passes to CUDA kernels, ensuring that once data is transferred to the GPU, it remains there for the duration of training.

In the hybrid version, some of the training steps were performed on the CPU, forcing intermediate results to be transferred between host and device memory. The GPU-native strategy eliminates these transfers by keeping all matrices resident in device memory.

1) *Latency of host-device memory transfers:* In a hybrid implementation, each CPU-GPU synchronization introduces delays that can overshadow kernel execution time, particularly when operations are fine-grained. This represents the primary bottleneck addressed by the GPU-native implementation.

The key idea is to treat the GPU as the primary compute device for the entire training loop. Once input data and model parameters are copied to device memory at the beginning of training, all subsequent operations are executed exclusively on the GPU.

This approach removes unnecessary host-device synchronization points and transforms the workflow into a fully device-resident pipeline, reducing latency and improving throughput.

2) *Implementation Details:*

- **Device-Resident Data:** Input dataset X , labels Y , weights W_1 and W_2 , and all intermediate tensors are allocated once in GPU global memory.
- **GPU-Based Forward Pass:** Matrix multiplications and activation functions are executed via CUDA kernels without copying intermediate outputs back to the CPU.
- **GPU-Based Backward Pass:** Gradient computations (MSE gradient, matrix multiplications for dW_1 and dW_2 , ReLU gradient) are performed entirely on the GPU.
- **On-Device Weight Updates:** Stochastic gradient descent updates are applied using a CUDA kernel, avoiding host-side parameter updates.

- **Minimized Data Transfers:** Host-device transfers occur only twice—once at initialization (loading data and weights) and once at the end (optional result retrieval).

3) *Expected Impact and Trade-offs:* This optimization significantly reduces training time by eliminating PCIe transfer overhead and repeated synchronization barriers. By keeping the full computational graph on the GPU, the training loop becomes a continuous stream of device-side operations, allowing the GPU scheduler to maintain higher utilization.

While full GPU offloading improves performance, it increases GPU memory usage because all intermediate tensors must remain allocated on the device. For very large models or datasets, memory capacity may become a limiting factor. Additionally, debugging becomes more complex, as intermediate values are no longer directly accessible from the CPU without explicit memory copies.

4) *Performance Analysis:* The GPU-native version was implemented as a baseline architectural optimization. All forward and backward operations were mapped to CUDA kernels, but each logical step (matrix multiplication, activation, gradient, update) was still executed as a separate kernel.

The training loop runs entirely on the GPU: each batch is processed using a sequence of CUDA kernels for forward propagation, loss gradient computation, backpropagation, and weight updates. No intermediate host-device transfers occur inside the loop.

Table IV compares the execution time of the hybrid (CPU+GPU) implementation and the fully GPU-native implementation. All experiments were conducted for 100 epochs with a batch size of 256.

TABLE IV
EXECUTION TIME COMPARISON: HYBRID VS GPU-NATIVE IMPLEMENTATION

Dataset	Samples	Hybrid Time (s)	GPU-Native Time (s)	Speedup
Small	256	0.3151	0.0187	16.85×
Medium	2560	3.6767	0.1752	20.98×
Large	25600	7.5937	1.3527	5.62×

The GPU-native implementation consistently outperforms the hybrid approach. The performance gain grows with dataset size, confirming that PCIe

transfer overhead becomes increasingly costly for larger workloads.

5) *Interpretation and Conclusion:* The full GPU offloading strategy establishes the GPU as the sole execution platform for the neural network training pipeline. By eliminating repetitive host-device memory transfers and synchronization points, the implementation achieves substantial reductions in runtime while preserving numerical correctness.

This optimization does not change the computational algorithm itself but improves system-level efficiency. It prepares the pipeline for further low-level optimizations by ensuring that all computations already reside on the GPU.

In conclusion, full GPU offloading is a fundamental architectural optimization that transforms a hybrid training workflow into a high-throughput device-resident pipeline, enabling the GPU to operate closer to its peak performance.

C. Kernel Fusion Optimization (GPU-Native)

After implementing a fully GPU-native version of the neural network training code, a new bottleneck emerges from the frequent invocation of small, independent kernels. In the unfused GPU-native implementation, each logical operation—matrix multiplication, activation function (ReLU), and gradient computation—is launched as a separate CUDA kernel. While this exposes parallelism, it incurs significant **kernel launch overhead** and unnecessary reads/writes to global memory.

Kernel fusion addresses these inefficiencies by merging multiple sequential operations into a single kernel. This allows intermediate results to remain in registers and shared memory, eliminating extra global memory transactions and reducing the number of kernel launches.

In the unfused GPU-native forward pass, each computation requires a separate kernel invocation. Kernel fusion merges the matrix multiplication and ReLU application into a single kernel. A similar mechanism is applied in the backward pass to avoid a separate ReLU gradient kernel.

1) *Kernel launch overhead:* The primary bottleneck addressed by kernel fusion is the **high kernel launch overhead** associated with multiple small GPU kernels. In the GPU-native code, separate kernels for each operation lead to redundant global

memory transactions and multiple synchronization points. For lightweight operations, the kernel launch overhead can dominate runtime, reducing the efficiency of the GPU.

The key idea of kernel fusion is to combine several dependent operations into a single CUDA kernel. This allows intermediate results to reside in registers or shared memory, minimizing global memory traffic. Fusion reduces kernel launch overhead and synchronization barriers, increases arithmetic intensity, and improves GPU utilization by keeping threads fully occupied.

2) Implementation Details:

- **Fused Forward Pass:** Each thread computes the dot product of the input and weight vector for one hidden unit and immediately applies the ReLU activation before storing it in global memory.
- **Fused Backward Pass:** The gradient through the ReLU activation is computed inline during the matrix multiplication with the output gradients, storing results directly in registers before writing to global memory.
- **Registers:** Intermediate accumulations are stored in thread-local registers.
- **Reduced Kernel Launches:** Multiple sequential kernels are replaced by a smaller number of fused kernels, cutting down kernel launch overhead.
- **GPU-Native Execution:** All fused computations are executed entirely on the GPU without host-side intervention between steps.

3) *Expected Impact and Trade-offs:* Fusing kernels is expected to significantly reduce training time by minimizing kernel launch overhead, reducing global memory transactions for intermediate results, and increasing instruction-level parallelism and arithmetic intensity.

Kernel fusion increases register usage and kernel complexity, which may limit occupancy. Excessive fusion can reduce modularity and make debugging more difficult. The benefits are most pronounced for small to medium-sized batches, where kernel launch overhead is a significant fraction of runtime. For large, memory-bound workloads, fusion alone may not yield substantial improvements without complementary memory optimizations.

4) *Performance Analysis*: The fused kernels were implemented on top of the GPU-native baseline, combining matrix multiplication with activation or activation gradient operations. No tiling was used; shared memory was not explicitly employed.

In the forward pass, each thread computes the dot product for one output element and immediately applies the ReLU activation. In the backward pass, the gradient with respect to the pre-activation is computed inline. This eliminates intermediate memory writes and synchronization points, directly addressing kernel launch overhead.

Table V summarizes the GPU execution times for defused and fused forward and backward passes on the synthetic convex datasets. All operations were executed on the GPU, and each experiment ran for 100 epochs with a batch size of 256.

TABLE V
GPU EXECUTION TIME COMPARISON BETWEEN DEFUSED AND FUSED KERNELS

Dataset	Defused (s)	Fused (s)	Speedup
Small	0.01827	0.0145	1.26×
Medium	0.1752	0.1045	1.68×
Large	1.3527	0.7267	1.86×

The table shows that the fused kernels consistently outperform the defused implementation. Speedup increases with dataset size, highlighting the growing impact of kernel launch overhead and memory traffic on larger workloads.

5) *Interpretation and Conclusion*: The kernel fusion strategy combines operations that were previously executed separately: `matmul + ReLU` in the forward pass and `matmul + ReLU gradient` in backpropagation. By fusing these operations, we achieve:

- **Reduced kernel launches**: Lower CPU-GPU synchronization overhead.
- **Improved memory efficiency**: Intermediate results are retained in registers/shared memory, reducing global memory access.
- **Better utilization of GPU parallelism**: Instructions are executed more efficiently.

Empirical results confirm that fusion leads to significant reductions in training time, particularly for medium and large datasets. Even for the small dataset, a $\sim 1.26\times$ speedup is observed, indicat-

ing that kernel launch overhead can have a non-negligible effect.

In conclusion, kernel fusion built on top of the GPU-native implementation is an effective optimization strategy that improves performance without altering the algorithm or model accuracy. This strategy provides a solid foundation for further optimizations, such as tiling or mixed-precision computation, especially for larger datasets.

D. Fully GPU-Resident Training with Forward Tiling and Kernel Fusion

At this stage, the proposed implementation combines **tiling** for matrix multiplications using shared memory and **selective kernel fusion** in the forward pass by merging the hidden-layer matrix multiplication with the ReLU activation. In addition, the backpropagation path is implemented with explicit CUDA kernels to ensure proper usage of row-major memory layout.

1) *Addressed bottleneck*: This strategy targets:

- **GPU-native execution**: keep the full training step (forward, gradients, SGD updates) on the GPU after the initial host-to-device transfer.
- **Memory access optimization**: reduce global memory traffic during dense linear algebra via tiling.
- **Reduced kernel count for activation**: fuse `matmul + ReLU` in the hidden layer to eliminate an extra activation kernel and intermediate global-memory write.

2) *Implementation Algorithm*: The below algorithm explains the general logic of the implementation:

3) *Expected Impact and Trade-Offs*: This approach is expected to improve performance by increasing arithmetic intensity in the forward `matmul` through tiling, as well as removing an intermediate kernel and global-memory write via kernel fusion.

However, despite selective fusion, each batch still launches several kernels (forward, loss gradient, gradient reductions, SGD), so launch overhead can dominate for very small workloads. Additionally, certain kernels (`grad_W1` and `grad_W2`) use direct reductions over the batch dimension; more optimized reductions or GEMM-based formulations could further improve performance.

Algorithm 1 GPU-Resident Training with Tiling and Kernel Fusion

Require: Dataset (X, Y) , batch size B , epochs E , tile size T

Ensure: Trained weights W_1, W_2

Initialize W_1, W_2 on GPU

for epoch $e = 1$ to E **do**

for each batch of size B **do**

Forward Pass:

$\text{matmul_relu_tiled}(X_{\text{batch}}, W_1, Z_1)$

$\text{matmul_tiled}(Z_1, W_2, \hat{Y})$

Backward Pass:

$\text{relu_backprop_output1}(dY, W_2, Z_1, dZ_1)$

$\text{grad_W1}(X_{\text{batch}}, dZ_1, dW_1)$

$\text{grad_W2}(Z_1, dY, dW_2)$

Weights Update:

$\text{sgd}(W_1, dW_1)$

$\text{sgd}(W_2, dW_2)$

end for

end for

Output: W_1, W_2

4) *Empirical Performance Evaluation (Tile $T = 16$):* We evaluated the end-to-end training time over 100 epochs for three synthetic convex datasets with batch size 256, using tile size $T = 16$. Table VI reports the measured training times.

TABLE VI

END-TO-END GPU TRAINING TIME USING TILE SIZE $T = 16$ (100 EPOCHS, BATCH=256)

Dataset	Samples	Time (s)
Small	256	0.0106
Medium	2560	0.1045
Large	25600	0.8401

5) *Interpretation of the strategy:* The results demonstrate that the GPU-native implementation with tiled forward computation and correct back-propagation achieves strong end-to-end performance and scales efficiently with dataset size. While the number of samples increases by $100\times$, the total training time increases by $\sim 79.3\times$, indicating that fixed per-batch overheads (kernel launch and synchronization) are amortized as the workload grows.

In particular, the large dataset exhibits the lowest time per processed sample, consistent with

improved GPU utilization when executing more batches per epoch. Overall, the $T = 16$ tiling configuration provides the best observed end-to-end training times for these shapes ($B = 256$, $I = 32$, $H = 256$, $O = 1$), while leaving additional optimization opportunities.

E. Performance Profiling

Performance profiling was conducted using the same experimental setup as the baseline performance profiling. A fused and tiled CUDA implementation was used for the forward pass, back-propagation, and parameter updates. The profiled use case was training on the large dataset (25600 samples.)

The total end-to-end training time is 0.57 s, corresponding to approximately 5.7 ms per epoch. Host-side analysis indicates that execution time is dominated by synchronization and waiting mechanisms, with `poll` (50.7%) and `pthread_cond_timedwait` (40.0%) accounting for the majority of OS runtime activity. This behavior reflects a CPU waiting for GPU completion rather than active computation, consistent with a GPU-bound workload.

CUDA API profiling shows that kernel launch overhead still represents the largest portion of API time, but it is amortized over the full training execution and does not significantly impact overall performance.

The distribution of GPU-time, represented in table VII confirms weight gradient computation and matrix multiplication as the primary computational bottlenecks.

TABLE VII

GPU KERNEL EXECUTION TIME BREAKDOWN

Kernel	Execution Time (%)
<code>grad_W1</code>	40.3
<code>matmul_tiled</code>	27.3
<code>grad_W2</code>	11.4
Activation backprop.	4.1
SGD update	3.9
MSE computation	1.9

Memory transfer overhead is minimal, with only four host-to-device transfers totaling 3.41 MB and a cumulative transfer time of 334 μ s. This confirms that data movement overhead is negligible compared to kernel execution time.

Overall, the profiling results demonstrate that the fused and tiled implementation achieves high computational efficiency, with low memory transfer overhead and minimal CPU involvement. These results provide a strong baseline for analyzing the impact of network size, number of neurons, and dataset scale on training performance.

V. REFERENCE CODE VS. OPTIMIZATION RESULTS

A. Reference Program Description

GPU Utilization: Matrix multiplications, which dominate computation, are performed on the GPU using a custom CUDA kernel. Each thread computes one element of the output matrix in parallel. For each multiplication:

- Input matrices are copied from host (CPU) to device (GPU) memory.
- The CUDA kernel executes parallel computations on the GPU.
- Results are copied back to host memory.

This leverages the GPU’s massive parallelism to accelerate the training process.

CPU Utilization: The CPU handles:

- Loading and batching the dataset,
- ReLU activation and its derivative,
- Loss calculation (Mean Squared Error),
- Weight updates and control flow.

By offloading matrix multiplication to the GPU, the program achieves significantly higher computational throughput compared to sequential execution. However, host-device memory transfers introduce some overhead.

B. Comparative Speedup Analysis

Table VIII compares the execution times of the reference parallelization strategy against our fully optimized solution combining the three strategies.

TABLE VIII
COMPARATIVE NEURAL NETWORK TRAINING TIMES (S).

Samples	reference code	optimized final code
256	0.3151	0.0106
2,560	3.6767	0.1045
25,600	7.5937	0.8401

The speedup factor ($\text{Speedup} = T_{\text{ref}}/T_{\text{opt}}$) highlights the dominance of the GPU approach:

- **Small Dataset:** $0.3151/0.0106 \approx 29.7\times$
- **Medium Dataset:** $3.6767/0.1045 \approx 35.2\times$
- **Large Dataset:** $7.5937/0.8401 \approx 9.0\times$

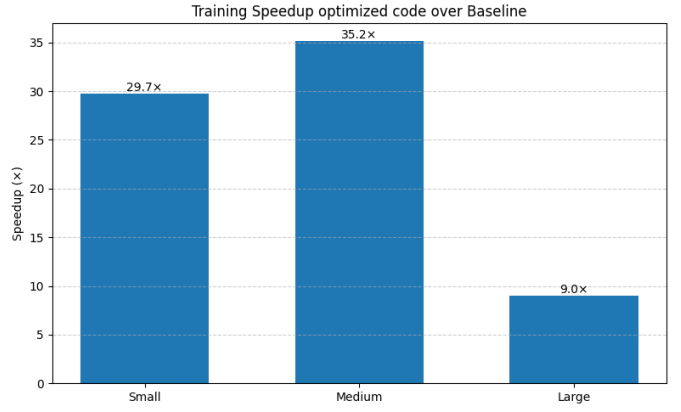


Fig. 7. Training speedup optimized coder over Baseline

C. Discussion

The results show that GPU acceleration combined with Fusion + Tiling optimizations provides superior scalability for neural network training. Execution time on the GPU grows sub-linearly with dataset size, whereas CPU-based methods scale nearly linearly, resulting in up to 9× speedup for the largest dataset. This demonstrates that memory-aware kernel fusion and tiling enable efficient utilization of GPU resources, maintaining high throughput as data scales. These findings highlight the importance of GPU-optimized pipelines for large-scale, parallelizable workloads.

Note on mixed results. While the fully integrated implementation (tiling + GPU-resident + selective fusion) yields large end-to-end speedups in most cases, we observed that the *GPU-native* + *fusion* variant (fusion without tiling) produced the best wall-clock time for the largest dataset in our runs. This indicates a non-trivial interaction between tiling and fusion: tiling increases arithmetic intensity but also raises per-block shared-memory / register usage which can reduce occupancy or create memory-bank / scheduling inefficiencies on the Tesla T4 for certain tile/shape combos. We therefore report per-dataset best-performing variants and discuss possible causes and remedies in the conclusion.

VI. STUDYING THE IMPACT OF HIDDEN LAYER SIZE

To understand how the size of the hidden layer affects training performance, we conducted experiments on three synthetic datasets (small, medium, and large) while varying the hidden layer size from 64 to 1024 neurons. The training was performed using the fully-fused tiled CUDA implementation described earlier.

As expected and illustrated by figure 8, training time increases with hidden layer size, with the effect becoming more pronounced for larger datasets due to the increased number of weight parameters and associated matrix operations.

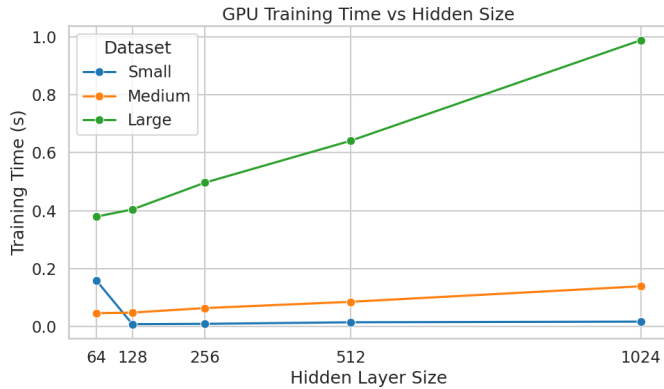


Fig. 8. GPU training time as a function of hidden layer size for small, medium, and large synthetic datasets.

This analysis highlights the computational cost of scaling the hidden layer in fully-connected networks and can guide the choice of network size for different dataset scales.

VII. EXPERIMENTAL SETUP

The table below presents the experimental setup used for the implementation and experimentation of the different optimisation strategies.

Component	Specification
Platform	Google Colab (Virtual Machine)
CPU	Intel Xeon @ 2.00 GHz
vCPUs	2 (1 core, 2 threads)
RAM	12 GB
GPU	NVIDIA Tesla T4
GPU Memory	15 GB
CUDA Version	12.8
GPU Driver	580.82.07
Virtualization	KVM (Full Virtualization)
Max threads per block	1024 threads

TABLE IX
HARDWARE AND SOFTWARE ENVIRONMENT USED FOR EXPERIMENTS

VIII. CONCLUSION AND FUTURE WORK

We presented a systematic study of three CUDA-centered optimizations for training a shallow neural network: (1) tiling for matrix multiplication, (2) full GPU offloading (keep data resident on-device), and (3) kernel fusion (merge consecutive operations into single kernels). Across three synthetic dataset scales (Small, Medium, Large) the optimized variants deliver substantial runtime reductions compared to the CPU / hybrid baseline. Figure 9 visualizes this progressive improvement across all optimization stages and dataset sizes.

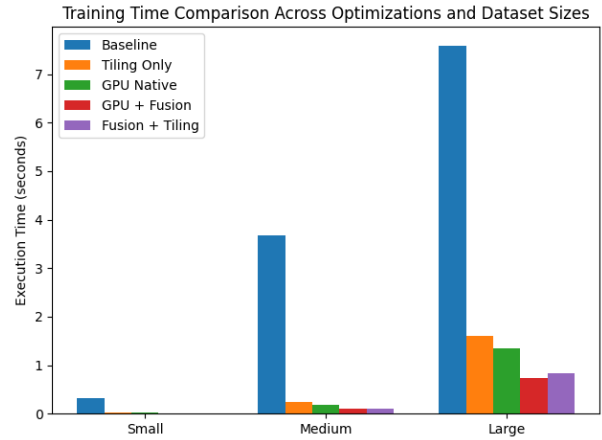


Fig. 9. Training time comparison across optimization strategies. Each successive optimization builds upon the previous, demonstrating cumulative performance improvements.

Key takeaways:

- **Strong overall gains.** Moving computation to the GPU and applying fusion and tiling reduces end-to-end training time by an order of magni-

tude or more across dataset sizes compared to the reference implementation.

- **No free lunch — variant-dependent best performer.** The *GPU-native + fusion* variant (fusion without tiling) consistently produced the best wall-clock time for the largest dataset in our experiments, while the fully integrated (tiling + fusion + GPU-resident) variant was the best for other sizes. This demonstrates that combining optimizations can produce non-linear interactions: tiling increases data reuse and arithmetic intensity but also increases shared-memory and register pressure which can reduce occupancy or worsen memory-bank behavior on a given GPU.
- **Performance drivers.** Profiling shows the reference implementation is dominated by host-device transfers and CUDA API overhead; after eliminating those, the remaining bottlenecks are kernel resource usage (registers / shared memory), per-kernel launch overhead and the efficiency of reduction kernels (e.g., gradient reductions).

Practical recommendations based on our findings:

- 1) Report per-dataset best variant and numbers (we do this in Section V). Do not claim a single method is universally best.
- 2) When applying tiling, tune tile size per problem shape (batch, input, hidden dims) and measure occupancy / register usage; the “largest” tile is not always best.
- 3) Use fusion selectively: fuse light-weight sequences to eliminate kernel-launch overhead, but inspect register pressure — over-fusion can reduce occupancy and hurt performance.
- 4) Always report both wall-clock (CPU+GPU) and pure GPU kernel times; they tell different stories (system-level vs kernel-level).

Future work

We plan the following investigations to explain and (if possible) remove the residual gap between variants:

- **Ablation / sensitivity study:** run controlled experiments varying a single factor (tile size, fusion degree, TPB) and produce heatmaps of time vs parameters.

- **Profile-driven tuning:** use Nsight Compute to collect per-kernel metrics (registers per thread, shared memory per block, achieved occupancy, memory throughput) and correlate them to run-time. Use these insights to choose tile sizes and fusion targets automatically.
- **Reduction optimizations:** replace custom reductions (gradients) with optimized GEMM or use hierarchical reduction strategies to reduce memory traffic and synchronization.
- **Mixed precision / Tensor Cores:** evaluate FP16 / mixed-precision training to leverage Tensor Cores on compatible GPUs for further speedup, while monitoring numerical stability.
- **Statistical reporting:** execute each experiment multiple times, report means and standard deviations, and present per-batch time distributions to quantify variance.

In summary, our results validate that GPU-residency and kernel fusion yield the largest single wins; tiling delivers further benefits when tuned carefully, but it must be combined with profiling-guided parameter selection to avoid hurting occupancy on a particular GPU. The honest reporting of per-dataset winners and the proposed follow-ups will make the manuscript stronger and more reproducible.

REFERENCES

- [1] Brouthen, K., & Akeb, A. (2025). *Exploring parallelization of shallow neural network using CUDA*. Student report, ESI.
- [2] NVIDIA Corporation. (2023). *CUDA C++ programming guide*. NVIDIA Developer Documentation. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- [3] NVIDIA Corporation. (2023). *NVIDIA Nsight Systems user guide*. NVIDIA Developer Documentation. <https://docs.nvidia.com/nsight-systems/>